

Language Translation at Scale 📄

Frank Aragona



the goal

- we need to automate writing a report or publication
- we're using markdown (quarto or rmarkdown)
- we need to translate the **text** (not the **code**)

steps to automatically translate

1. extract the text from the `.qmd/.rmd` file
2. apply translation model to the extracted text
3. insert the new text back into the `.qmd/.rmd`

step 1: extract the text

- lightparser R package
- distinguishes the code chunks from the text
- turns your `.qmd/.rmd` into a dataframe

```
1 use("lightparser")
2
3 (parsed ← lightparse
```

```
1 # A tibble: 8 × 8
2   type      label
3   <chr>    <chr>
4 1 yaml     <NA>
5 2 inline   <NA>
6 3 heading  <NA>
7 4 inline   <NA>
8 5 heading  <NA>
9 6 inline   <NA>
10 7 block    unnamed-chu
11 8 inline   <NA>
```

step 2: translate

- can use whichever translation api
- huggingface transformers
- huggingfaceR package
- translatemd package for this demo

```
1 parsed_es ← parsed |
2   tidyr::unnest(cols
3   dplyr::mutate(text_
4     purrr::map(text,t
5   )
```

```
1 # A tibble: 6 × 2
2   type      text_es
3   <chr>     <chr>
4 1 heading # Quarto
5 2 inline  Quarto le p
6 3 inline  Para crear
7 4 inline  Commits con
8 5 inline  - La palab
9 6 inline  - y subirá
```

step 3: convert back to quarto

- lightparser
- convert the new dataframe into a `.qmd/.rmd` file
- quarto render
- . . .

```
1 # output to qmd
2 lightparser::combine_
3   parsed_es_to_qmd,
4   "_spanish.qmd"
5 )
```

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

To create the release cycle in your repo you may want to use Conventional Commits.

Conventional Commits are a way to format and standardize your commit messages, which can be used to then automate the repo's release cycle. For example, one conventional naming method is to label any commit associated with a new feature as `feat:` plus a commit message.

- The word `feat:` can trigger a Github Action to add that commit to your changelog under the **Features** header,
- and it will up-version the minor release version number.
- So if you are on release 1.0.0, a new `feat` will up-version the cycle to 1.1.0
- Commit titles that start with the word `fix:` as in a bug fix will up-version the patch number of the, i.e. 1.0.0 to 1.0.1

Automating The Release Cycle

You should consider automating your release cycle so that your project cycle is consistent and predictable. There are many different ways to approach this.

Some repos have semi-automatic cycles where there is some manual component of releasing their software, whereas others are fully automated. Manual releases can work too for some scenarios.

Here's a code chunk

```
mtcars |>
  select(mpg)|>
  slice(1)
```

Quarto

Quarto le permite entretener el contenido y el código ejecutable en un documento terminado. Para obtener más información sobre Quarto, consulte <https://quarto.org>.

Para crear el ciclo de lanzamiento en su repo es posible que desee utilizar Commits convencionales.

Commits convencionales son una forma de formatear y estandarizar sus mensajes de commit, que se puede utilizar para automatizar el ciclo de lanzamiento de la repo. Por ejemplo, un método convencional es etiquetar cualquier commit asociado con una nueva característica como `feat:` más un mensaje de commit.

- La palabra `feat:` puede activar una acción de Github para añadir que se compromete a su

registro de cambios bajo el encabezado **Características**,

- y subirá el número de versión de lanzamiento menor.
- Así que si usted está en la versión 1.0.0, un nuevo `feat` subirá la versión del ciclo a 1.1.0
- Commit títulos que comienzan con la palabra **fix:** como en una corrección de errores subirá la versión del número de parche del, es decir, 1.0.0 a 1.0.1

Automatizando el Ciclo de Liberación

Usted debe considerar automatizar su ciclo de lanzamiento para que su ciclo de proyecto sea consistente y predecible. Hay muchas maneras diferentes de abordar esto.

Algunas repos tienen ciclos semiautomáticos donde hay algún componente manual de la liberación de su software, mientras que otras están completamente automatizadas. Las releases manuales también pueden funcionar para algunos escenarios.

Aquí hay un trozo de código.

```
mtcars |>
  select(mpg)|>
  slice(1)
```


more ideas?

- text to speech
- text summarization
- ?

links

- blog
- presentation repo

